Contents

Mixed Precision Allocation Under Visual Distribution Shift — Project Deep Dive 1
What changed in this revision . . . 1

Part 1 — The project in simple English 2
1.1 The thing being studied . . . 2
1.2 What people do today, and why it might be wrong . . . 2
1.3 Why it plausibly goes wrong . . . 2
1.4 The measurement . . . 3
1.5 The fairness rule . . . 3
1.6 The data . . . 3

Part 2 — Deep dive 4
2.1 Architecture and where the weight actually lives . . . 4
2.2 The eight configurations: measured . . . 4
2.3 The quantizer, and a decision that must be frozen . . . 6
2.4 Selection procedure . . . 7
2.5 Statistics . . . 7
2.6 Threats to validity . . . 7
2.7 Current state . . . 9
2.8 Code review findings from this revision . . . 9

Part 3 — Specifications 9
3.1 The hard requirement . . . 9
3.2 Validated configuration . . . 9
3.3 Minimum and recommended . . . 10
3.4 Disk breakdown . . . 10
3.5 Compute budget — revised upward . . . 10
3.6 If hardware is the constraint . . . 11

Mixed Precision Allocation Under Visual Distribution Shift — Project Deep Dive

Analysis document. Revised 2026-10-29. This is not the protocol and not a results document. The authoritative specification is PROPOSAL_SNAPSHOT.md; protocol changes belong in DECISIONS.md; current state lives in live_STATUS.md.

Every number is labelled measured, derived, or estimated. No task-accuracy experiment has been run. There is still no Δ result.

What changed in this revision

The first revision of this document (2026-10-22) predicted per-configuration memory from the published model config while NF4 was still blocked on hardware. The CUDA audit has since completed on a Colab Tesla T4, so this revision replaces predictions with measurements:

Area	First revision	Now
M1 component control	unproven, flagged as the project's main blocker	measured and accepted — 331 linear modules, zero mismatches
Per-config memory	derived from config.json	measured on a T4; predictions held to 0.09%
Connector size	21.2 M params (derived)	confirmed, and it is exactly one linear module
Compute budget	estimated 15–40 GPU-hours	revised upward — the only measurement implies up to ~106

Area	First revision	Now
Caps	predicted four tight pairs	confirmed, exact byte thresholds now known

Two new issues are raised here that were not visible before the audit: the cross-domain blur-radius confound (§2.6, threat 3) and the double-quantization decision (§2.3).

Part 1 — The project in simple English

1.1 The thing being studied

A vision-language model (VLM) answers questions about pictures. The one in this study, SmolVLM2-2.2B, has three working parts:

Part	Job	Analogy
Vision encoder	Looks at the image and turns it into numbers	The eye
Connector	Translates those numbers into the language model's vocabulary	The optic nerve
Language backbone	Reads the translation and writes the answer	The brain

Quantization makes a model smaller by storing its weights with fewer bits. Normally each weight is a 16-bit number. Store it in 4 bits instead and that part of the model takes about a quarter of the space — but the numbers get rounded, so the model gets slightly worse at its job.

The useful fact: you don't have to compress all three parts equally. You can squeeze the language backbone to 4 bits and leave the eye at full precision, or the other way round. With two choices (4-bit or 16-bit) for each of three parts, there are $2 \times 2 \times 2 = 8$ possible configurations.

1.2 What people do today, and why it might be wrong

If you want to deploy one of these models on a phone or a small device, you have a memory limit. So you try the configurations that fit, test them on some images, and ship whichever scored best.

The images you test on are almost always clean, well-lit, in-focus photos — because those are the images that come with benchmarks.

The images your model actually sees in the field are often not clean. A security camera at night. A phone photo through a dirty lens. A warehouse in bad light.

So: is the configuration you picked using clean photos still the right configuration once the real images are dark or blurry?

Nobody has checked. That's the project.

1.3 Why it plausibly goes wrong

Here is the mechanism that makes this more than a bookkeeping exercise.

Blur destroys fine detail in an image. Quantization adds rounding noise. Both degrade the same signal — the visual information flowing from the eye, through the optic nerve, into the brain. They might compound.

If they do, the best configuration could change depending on image quality:

- On clear photos, the vision encoder has a strong, clean signal to work with, so rounding its weights costs little. The sensible move is to protect the language backbone.
- On blurry photos, the visual signal is already weak. Now rounding the weights that process it might be the thing that breaks the answer — so protecting the vision side could matter more.

Same memory budget, different right answer. If that's true, then everyone tuning compression on clean benchmark images is shipping a misconfigured model.

1.4 The measurement

The whole study reduces to one subtraction. Build a test set of dark and blurry images. Then pick a configuration two different ways:

- Way A (what people do): pick the configuration that scores best on clean images.
- Way B (the ideal): pick the configuration that scores best on dark and blurry images.

Run both on the same dark/blurry test set:

$$\Delta = (\text{accuracy of the config picked way B}) - (\text{accuracy of the config picked way A})$$

- Δ is large and positive → picking on clean images cost you real accuracy. Current practice is wrong, and you've measured by how much.
- Δ is about zero → the choice travels fine. Calibrate on whatever is convenient. This confirms an assumption the field currently makes with no evidence behind it.

Both outcomes are publishable. This is the single best feature of the design. Most research projects fail if the effect isn't there; this one answers its question either way. A null result is a real contribution, provided the confidence intervals are tight enough to rule out an effect that would matter.

There's a third, practical option too: pick the configuration using a mixture — mostly clean images with 25%, 50%, or 75% degraded ones. If a small amount of realistic data recovers most of the loss, that's directly actionable advice for practitioners, who can usually collect a few hundred representative images but not a few hundred thousand.

1.5 The fairness rule

Comparing a 4.5 GB model against a 1.5 GB model proves nothing — of course the bigger one wins. So every comparison happens under a memory cap: only configurations that fit the same budget compete against each other.

This is what turns the question from "does compression hurt" (obviously yes) into "given a fixed budget, where should you spend your bits" — which is the question an engineer actually faces.

1.6 The data

Two datasets, chosen deliberately to look nothing like each other:

Dataset	Content	Question style
POPE-COCO	Everyday scenes and objects	"Is there a dog in this image?"
CelebA	Cropped human faces	"Is this person wearing glasses?"

Both are yes/no, so scoring is unambiguous — no judging free-text answers. And because the two datasets share almost no visual statistics, if the transfer penalty differs between them, that difference is attributable to image content rather than to overlap.

Then each image is used in seven conditions: the original, plus three levels of darkening and three levels of blur. Same image, same question, same correct answer — only the picture quality changes. That pairing is what makes the comparison clean.

Part 2 — Deep dive

2.1 Architecture and where the weight actually lives

Component sizes derived from the published `config.json` for `HuggingFaceTB/SmolVLM2-2.2B-Instruct` (revision `482adb53…`): a SigLIP-SO400M/14 @384 vision tower, a pixel-shuffle MLP connector (`scale_factor=3`), and a SmolLM2-1.7B text backbone.

Component	Params	Share	FP16	Linear modules (measured)
Vision encoder (linear layers)	411.1 M	18.3%	822 MB	162
Vision encoder (patch/pos embeddings)	1.5 M	0.1%	3 MB	—
Connector	21.2 M	0.9%	42 MB	1
Text backbone (linear layers)	1,610.6 M	71.7%	3,221 MB	168
Text embedding	100.9 M	4.5%	202 MB	—
`lm_head` (FP16 exception)	100.9 M	4.5%	202 MB	—
Total	2,246.3 M	100%	4,493 MB	331

The FP16 total is measured at 4,493,569,888 bytes — identical on CPU (macOS) and on the T4, and matching this derivation to within 0.02%.

The connector problem

The connector is 0.9% of the weights and exactly one linear module. This is the most important structural fact in the project.

Quantizing it saves a measured 30,523,328 bytes — 0.7% of the model. So in any accuracy-versus-memory plot, the connector's precision is nearly free. That cuts three ways:

- Bad news: the connector cannot be a meaningful lever for hitting a memory cap. You will never need to quantize it to fit a budget.
- Good news: it makes the central hypothesis cheap to test. If blur damage and quantization noise really do compound at the connector, you get that finding for 30 MB — a precision decision with negligible memory cost and a measurable accuracy consequence.
- The new caveat: because it is a single layer, a null connector result is ambiguous. "Connector precision does not matter" and "one 21 M-param layer is too small a lever to show an effect" predict the same measurement. Say so in the write-up rather than claiming the stronger reading.

If connector precision shows any accuracy benefit under shift, the practical recommendation is simply "always keep the connector at FP16" — a concrete deployment rule, but a conclusion about accuracy only, not about the memory trade-off.

2.2 The eight configurations: measured

Measured on a Colab Tesla T4, all eight loaded freshly from the same checkpoint. Weight bytes count unique parameter storage, persistent buffers, and exposed quantization metadata. Peak reserved is a separate CUDA measurement from one synthetic 384×384 inference at batch size 1.

Vision	Conn.	Backbone	Weight bytes	GiB	Peak reserved GiB	Gen. s
NF4	NF4	NF4	1,556,898,080	1.450	2.283	3.98
NF4	FP16	NF4	1,587,421,408	1.478	2.283	4.05
FP16	NF4	NF4	2,147,801,504	2.000	2.600	3.90

Vision	Conn.	Backbone	Weight bytes	GiB	Peak reserved GiB	Gen. s
FP16	FP16	NF4	2,178,324,832	2.029	2.627	3.89
NF4	NF4	FP16	3,872,143,136	3.606	4.182	3.98
NF4	FP16	FP16	3,902,666,464	3.635	4.213	3.85
FP16	NF4	FP16	4,463,046,560	4.157	4.781	4.01
FP16	FP16	FP16	4,493,569,888	4.185	4.957	3.78

All eight produced a successful deterministic generation. This checks execution viability only — it is not accuracy, robustness, real-image, or throughput evidence.

Prediction versus measurement

The first revision of this document derived these totals before any GPU was available. Every configuration landed within 1.4 MB:

Candidate	Derived	Measured	Error
NF4/NF4/NF4	1,556 MB	1,556.9 MB	−0.058%
NF4/FP16/NF4	1,586 MB	1,587.4 MB	−0.090%
FP16/NF4/NF4	2,147 MB	2,147.8 MB	−0.037%
FP16/FP16/NF4	2,177 MB	2,178.3 MB	−0.061%
NF4/NF4/FP16	3,871 MB	3,872.1 MB	−0.030%
NF4/FP16/FP16	3,902 MB	3,902.7 MB	−0.017%
FP16/NF4/FP16	4,462 MB	4,463.0 MB	−0.023%
FP16/FP16/FP16	4,493 MB	4,493.6 MB	−0.013%

This matters beyond bookkeeping: the storage model is now validated, so memory consequences of future candidate changes (an 8-bit level, a different model size) can be computed before spending GPU time.

Component savings are exactly additive

Measured, holding the other two components at FP16:

Component moved to NF4	Bytes saved
Vision encoder	590,903,424
Connector	30,523,328
Language backbone	2,315,245,056

```
4,493,569,888 − 590,903,424 − 30,523,328 − 2,315,245,056 = 1,556,898,080
```

That is exactly the measured all-NF4 total. Perfect additivity is the strongest evidence in the audit that the three component controls are independent and do not contaminate each other — the single most dangerous failure mode for this study, now ruled out at the storage level.

Cap structure

The eight candidates form four tight pairs; within each pair only the connector differs (~30 MB), while vision costs ~591 MB and the backbone ~2,315 MB. Exact measured thresholds:

Cap (bytes)	Eligible	Vision varies	Backbone varies	Use as a primary cap?
1,556,898,080	1	no	no	no — single candidate
1,587,421,408	2	no	no	no — connector-only comparison
2,147,801,504	3	yes	no	usable
2,178,324,832	4	yes	no	yes
3,872,143,136	5	yes	yes	usable
3,902,666,464	6	yes	yes	yes
4,463,046,560	7	yes	yes	no — barely constrains
4,493,569,888	8	yes	yes	no — equals FP16 size

The M1 acceptance criterion asked only for a cap with two distinct candidates, and 1,587,421,408 satisfies it. But it is the weakest possible scientific choice, because its two candidates differ only in connector precision. The audit report flags this itself.

Recommendation: freeze 2,178,324,832 (four candidates, vision and connector vary at NF4 backbone) and 3,902,666,464 (six candidates, backbone varies too) as the primary caps. Both must still be confirmed during the pilot without consulting comparative test accuracy.

2.3 The quantizer, and a decision that must be frozen

The study uses bitsandbytes NF4: measured policy is NF4 storage, FP16 compute, uint8 quantized storage, explicit FP16 component-root exclusions, an FP16 `lm_head` exception, and no double quantization.

The metadata overhead confirms the scheme exactly. For the all-NF4 candidate:

Storage category	Bytes
Parameter storage	1,429,194,464
Persistent buffers	128
Exposed quantization metadata	127,703,488
Total	1,556,898,080

127,703,488 bytes over 2,042.9 M quantized params is 0.0625 bytes/param — exactly one FP32 `absmax` per 64-value block. The storage model is fully accounted for.

> Open decision. Enabling double quantization would shrink that metadata from ~128 MB to ~16 MB and therefore shift every cap in §2.2. It is currently off. Freeze this in DECISIONS.md before the pilot; changing it mid-study invalidates cross-run comparisons. Consistency matters more here than the 112 MB.

Why NF4 makes the result cleaner than the earlier design

An earlier version of this proposal used GPTQ or AWQ, which are calibration-dependent: they examine sample data and adjust the quantized weights accordingly. The final design uses NF4, which is data-free — the quantized weights are a deterministic function of the original weights alone.

This is a genuine improvement in experimental hygiene. With a calibration-dependent quantizer, a positive Δ has two causes that are hard to separate: the configuration choice was wrong, or the calibration data was unrepresentative. With NF4 there is no calibration step, so a positive Δ can only come from the configuration choice. The confound is designed out rather than controlled for.

The cost is that the study no longer speaks to calibration-set selection, which is what most of the related literature is about. Cite that literature as motivation — methods like MODE, SPEED-Q, and MASQuant all measure component sensitivity on a sample and allocate bits from it, and none test whether the measurement transfers — while being explicit that this study tests a cleaner, narrower version of the question.

2.4 Selection procedure

At each memory cap with at least two feasible configurations:

1. Score every feasible configuration on the development set.
2. Do this three ways: clean-only, target-only, and mixed (25%, 50%, 75% target).
3. Take the argmax for each. Break ties toward lower memory, then by a fixed predetermined order.
4. Evaluate the selected configurations on the untouched target test set.
5. Δ = target-selected accuracy − source-selected accuracy, in percentage points.

Four invariants make this valid, and each deserves a test:

- Equal budgets. Every selection strategy sees the same number of development examples and the same number of distinct images. Otherwise a positive Δ could just mean "more data wins."
- Common candidate set. All strategies choose from the same feasible configurations at a given cap.
- Fixed everything else. Prompts, image resolution, decoding settings, and answer parsing are frozen before selection begins.
- Test set never consulted. Both Δ terms come from development data. The selector must be structurally incapable of reading test accuracy — enforce it in code, not by discipline.

2.5 Statistics

Paired cluster bootstrap, resampling by original image (by identity for CelebA), keeping all questions and all seven corrupted variants of an image together in whatever resample it lands in.

Why clustering is mandatory: the seven conditions of one image are not seven independent observations. They share a subject, a scene, and a correct answer. Treating them as independent would shrink the confidence intervals by roughly the square root of the cluster size and manufacture significance that isn't there. The same applies to CelebA identities, where one person appears in many images.

Why pairing matters: both Δ terms are measured on the same test items, so the comparison is within-item. Bootstrapping the difference directly is far more powerful than comparing two independently-bootstrapped accuracies, because the item-difficulty variance cancels.

Selection stability is a separate analysis from Δ. Resample the development set and re-run selection; record how often each configuration wins. If the winner flips constantly across resamples, Δ is measuring noise in the selection procedure rather than a real property of the model — and that finding must be reported, not buried.

Reporting a null honestly means stating which effect sizes the study can exclude. "No significant difference" with an interval of ±8 points says almost nothing. The same statement with ±1.5 points is a real invariance result. This decides whether a null outcome is a contribution or a failed experiment, so power should be sanity-checked at the pilot stage.

2.6 Threats to validity

#	Threat	Status	Mitigation
1	Component control doesn't work	Resolved. 331/331 modules converted as requested, zero mismatches, 8 distinct signatures, savings exactly additive	Audit is re-runnable; keep it as a gate whenever the candidate grid or library versions change

#	Threat	Status	Mitigation
2	Throughput	Open, now the top risk. Only measurement is 3.78–4.05 s/answer (cold, includes warm-up)	Measure steady-state in the pilot before committing. See §3.5
3	Blur radius is not comparable across domains	Open, newly identified. Gaussian radius is in absolute pixels, so one radius is relatively far stronger on a ~178 px CelebA crop than on a ~640 px COCO image	Define severity relative to image size (radius as a fraction of the short edge), or document that severities are per-domain and never compare raw severity across domains. A protocol decision, not a code fix
4	Severities miss the useful window	Open	The 100-question pilot must show graded degradation — accuracy declining with severity but not collapsing to chance. Pick severities on pilot data only
5	Split leakage	Mitigated	Split by original image before generating variants; CelebA by identity. Checksum-linked groups now merged so identical pixels under two IDs cannot straddle splits. 42 tests pass
6	Connector result is ambiguous	Open by construction	One layer; a null is not evidence of no effect (§2.1)
7	Selection noise swamps the effect	Open	Report selection frequencies alongside Δ; size the dev set from pilot variance
8	Invalid answers smuggle in bias	Open	Count invalid as incorrect, fix the parser before selection, report invalid rate per configuration, keep infrastructure failures distinguishable from model answers
9	8 GB ceiling	Largely resolved	FP16 reference peaked at 4.957 GiB with one synthetic image. Re-check with real images; weight bytes and peak GPU memory stay reported separately
10	Double quantization changes every cap	Open decision	Freeze before the pilot (§2.3)

2.7 Current state

Milestone	Status
M0 — environment, model revision pinned	Done, macOS + CUDA
M1 — component control + audit	Accepted — CUDA/NF4-audited on a T4
M2 — data and corruption pipeline	Implemented; POPE validated on real data, CelebA annotations validated, images not downloaded
M3 — pilot, prediction cache, protocol freeze	Not started
M4 — selection and statistics	Not started
M5 — main runs and report	Not started

M2 real-data validation, Phase A. POPE-COCO is fully checked against the three official COCO files and the 500 referenced val2014 images: 3,000 questions, 500 images, 1,500/1,500 yes/no, 6 questions per image, no duplicate IDs, no missing images, deterministic normalization. CelebA annotation files match published MD5s — 202,599 images, 40 attributes, 10,177 identities — but the images themselves are not yet downloaded. 615 identities carry inconsistent `Male` labels; that is dataset label noise to document, not to repair.

Trial splits showed zero leakage and byte-identical reruns; trial corruptions produced 420 variants with zero pixel mismatches across two renders.

Still to decide (not frozen): the POPE sampling variant, the CelebA attribute set, the identity-capped balanced sampling rule, split sizes, the seed, corruption severities, and the primary caps.

2.8 Code review findings from this revision

Reviewed `splits.py`, `corruptions.py`, `records.py`, `memory.py`, and `quantization.py`. The split logic is sound: `group_id` resolves to the CelebA identity and the POPE original image, so protocol-level grouping is correct by construction, and the union-find that merges groups sharing an image checksum correctly picks the lexicographically smallest member as representative, so data without shared checksums keeps its previous assignment.

One defect was found and fixed in this revision:

> `CorruptionSpec.value` was not normalized, so `2` and `2.0` hashed to different `signature` values, and `True` was accepted as a blur radius (`bool` is an `int` subclass, and `True <= 0` is false). Three spellings of one transform produced three cache keys. Since `AGENTS.md` requires cache keys to include the image transformation, this would have caused silent cache misses and inconsistent manifest immutability comparisons. `value` is now coerced to `float` and `bool` is rejected, with two regression tests.

Part 3 — Specifications

3.1 The hard requirement

An NVIDIA GPU with CUDA. bitsandbytes NF4 has no Apple Silicon or CPU backend. Apple Silicon is usable for development only: code, unit tests, module audits, and FP16 CPU loading.

This is now confirmed in both directions — the macOS host reported no CUDA, no bitsandbytes, and `mps_available=False`, while the full eight-candidate audit succeeded on a T4.

3.2 Validated configuration

The measured environment in which the audit succeeded:

Item	Value
GPU	Tesla T4, 15,360 MiB, compute capability 7.5
Driver	580.82.07
CUDA runtime (PyTorch)	13.0
PyTorch	2.11.0+cu130
Transformers	4.52.4 (5.20.1 breaks SmolVLM image-token merging)
bitsandbytes	0.50.2
Accelerate	1.15.0
Python	3.13.16

A free-tier Colab T4 is sufficient for the audit and is the cheapest known working path.

3.3 Minimum and recommended

Resource	Minimum	Recommended
GPU	NVIDIA 8 GB, CC ≥ 7.5 (T4 / RTX 2060+)	16–24 GB
System RAM	16 GB	32 GB
Disk	25 GB free	50 GB SSD
CPU	Any modern 4-core	—
OS	Linux preferred	Linux

The 8 GB floor is confirmed viable: the largest configuration peaked at 4.957 GiB reserved, leaving ~3 GB headroom on an 8 GB card. That measurement is for one 384×384 image, a short prompt, and four generated tokens — real POPE prompts with more image tokens will push higher, so re-check during the pilot. Compute capability 7.5 is confirmed as sufficient, not merely assumed.

3.4 Disk breakdown

Item	Size	Notes
SmolVLM2-2.2B checkpoint	~4.5 GB	FP16 safetensors
COCO val2014 images	~6.3 GB	Or ~80 MB for only the 500 POPE-referenced images
CelebA aligned images	~1.4 GB	Research-use agreement; stays internal
Corrupted variants	0–2 GB	Zero if generated deterministically on the fly — the better choice
Prediction cache	~100 MB	~95k JSON records
Python environment	~6 GB	PyTorch and CUDA libraries dominate
Total	~19–21 GB	25 GB minimum

3.5 Compute budget — revised upward

Derived answer counts:

Run	Count	Arithmetic
Test	56,000	500 questions × 7 conditions × 8 configs × 2 domains

Run	Count	Arithmetic
Development (illustrative at 300 q/domain)	33,600	300 × 7 × 8 × 2
Pilot	~5,600	100 × 7 × 8
Total	~95,000	

The first revision of this document estimated 0.3–1.0 s per answer and 15–40 GPU-hours. That estimate had no measurement behind it and was too optimistic. The audit's generation times are 3.78–4.05 s:

s/answer	95,000 answers
4.0	105.6 GPU-hours
2.0	52.8
1.0	26.4

The audit correctly labels its timings as cold single-inference measurements including load and warm-up, not a throughput benchmark, so steady-state is probably materially faster. But the honest planning position is that the budget is somewhere between 26 and 106 GPU-hours and nobody knows where. Establishing steady-state throughput is therefore the pilot's single most important job, ahead of severity selection.

At the pessimistic end this does not fit free-tier Colab, whose sessions disconnect and whose GPU quota is limited. Budget for a paid runtime or a local card, and treat resumable caching as mandatory rather than an optimization.

Two multipliers worth protecting:

- Caching is what makes this tractable. Each (config, condition, question) triple is inferred once and stored. Every memory cap, mixture ratio, and bootstrap resample is then recomputed from the cache with no GPU involvement.
- Cache keys must be complete — model and processor revision, actual audited quantization state, image transformation, prompt, and decoding settings. An incomplete key silently serves one configuration's answers for another. The `CorruptionSpec.signature` defect in §2.8 was exactly this class of bug.

3.6 If hardware is the constraint

Fallbacks, in order of increasing scientific cost:

1. Reduce severity count from 3 to 2 per corruption → 5 conditions instead of 7, a 29% cut.
2. Exploit the pair structure. If connector precision proves accuracy-neutral in the pilot, the eight candidates collapse to four and the budget halves. This must be established empirically, not assumed.
3. Drop to SmolVLM2-500M. Same architecture family, so the three-component structure is preserved. Conclusions narrow to the smaller model.
4. Rent a GPU. Tens of GPU-hours on a 24 GB card is inexpensive and removes the schedule risk that free-tier session limits introduce.